

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA - CEFET/RJ *CAMPUS* PETRÓPOLIS
CURSO: BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

BRKGA Aplicado ao Problema de Sequenciamento de Tarefas em Ambientes Industriais com Múltiplos Recursos

Gabriel Cardoso Nascimento

PETRÓPOLIS
2025

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA - CEFET/RJ *CAMPUS* PETRÓPOLIS
CURSO: BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

BRKGA Aplicado ao Problema de Sequenciamento de Tarefas em Ambientes Industriais com Múltiplos Recursos

Gabriel Cardoso Nascimento

Trabalho de Conclusão de Curso apresentado ao Cefet/RJ - *campus* Petrópolis, como parte dos requisitos para obtenção do título de Bacharel em Engenharia de Computação.

Orientadora: Prof.ª Laura Silva de Assis

PETRÓPOLIS
2025

Autorizo a reprodução e divulgação total ou parcial deste trabalho, por qualquer meio eletrônico ou convencional, para fins de estudo e pesquisa, desde que citada a fonte.

Cefet/RJ – Sistema de Bibliotecas / Uned Petrópolis

N244b Nascimento, Gabriel Cardoso.

BRKGA Aplicado ao Problema de Sequenciamento de Tarefas em Ambientes Industriais com Múltiplos Recursos / Gabriel Cardoso Nascimento. - 2025.

33f.: il.

Trabalho de conclusão de curso (Curso de Bacharelado em Engenharia de Computação) – Centro Federal de Educação Tecnológica Celso Suckow da Fonseca, Petrópolis (RJ), 2025.

Bibliografia: p. 32 - 33.
Orientadora: Prof.ª Laura Silva de Assis, D.Sc.

1. Otimização Combinatória. 2. Escalonamento de Tarefas. 3. Máquinas Paralelas. 4. Metaheurísticas. 5. Biased Random-Key Genetic Algorithm (BRKGA). 6. Makespan. 7. Monografia. I. Assis, Laura Silva de. (Orienta.). II. Título.

CDD 519.64

Elaborada por Luciana de Souza Castro CRB7/ 4812

CEFET/RJ
campus Petrópolis

CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA - CEFET/RJ *CAMPUS* PETRÓPOLIS
CURSO: BACHARELADO EM ENGENHARIA DE COMPUTAÇÃO

FOLHA DE APROVAÇÃO

Otimização de algoritmos para ordenamento de tarefas em ambientes industriais com múltiplos recursos

Gabriel Cardoso Nascimento

Trabalho de Conclusão de Curso apresentado ao CEFET/RJ - *campus* Petrópolis, como parte dos requisitos para obtenção do título de Bacharel em Engenharia de Computação.

Orientadora: Prof.ª Laura Silva de Assis

Aprovado por:

LAURA SILVA DE ASSIS:06571252619
Assinado de forma digital por LAURA SILVA DE ASSIS:06571252619
Dados: 2025.07.28 14:15:52 -03'00'

Prof.ª Laura Silva de Assis, D.Sc. (Orientadora)

LUIS DOMINGUES TOME JARDIM TARRATACA:70567312100
Digitally signed by LUIS DOMINGUES TOME JARDIM TARRATACA:70567312100
Date: 2025.07.28 21:04:18 +01'00'

Prof. Luis Domingues Tome Jardim Tarrataca, Ph.D.

gov.br
Documento assinado digitalmente
PEDRO HENRIQUE GASPARETTO LUGAO
Data: 28/07/2025 17:42:11-0300
Verifique em https://validar.iti.gov.br

Prof. Pedro Henrique Gasparetto Lugão, D.Sc.

Janeiro de 2025

DEDICATÓRIA

Dedico este trabalho, em primeiro lugar, à minha família, cujo apoio foi fundamental desde o início dessa jornada. Sem o incentivo, o carinho e a confiança de vocês, eu não teria conseguido dar os primeiros passos nem manter a motivação nos momentos mais difíceis. Dedico também a mim mesmo, pelo esforço, pela persistência e por continuar mesmo diante das incertezas e desafios ao longo do caminho.

AGRADECIMENTO

Agradeço à Prof.ª Laura pela orientação ao longo deste trabalho, pela indicação do caminho metodológico, das ferramentas, e por me guiar com clareza e atenção. Agradeço também aos professores do curso, cuja contribuição foi essencial para minha formação acadêmica e profissional.

RESUMO

Resolver o problema de sequenciamento de tarefas em ambientes industriais é um problema de otimização combinatória notoriamente complexo, classificado como NP-difícil. Essencialmente, o problema de sequenciamento de tarefas, também conhecido como programação de tarefas, envolve a definição da ordem ideal para executar um conjunto de atividades, considerando recursos limitados e restrições, de modo a otimizar um determinado objetivo. Este trabalho propõe e avalia uma solução baseada na meta-heurística *Biased Random-Key Genetic Algorithm* (BRKGA) para resolver o problema de sequenciamento de tarefas em máquinas paralelas idênticas com restrições de precedência, visando a minimização do tempo total de produção (*makespan*). A metodologia desenvolvida se destaca pelo uso de um decodificador customizado. Esse componente traduz as chaves aleatórias do cromossomo em uma lista de prioridades de tarefas, alocando-as de forma eficiente por meio de uma heurística construtiva baseada no critério de Menor Tempo de Término (*Earliest Finish Time* - EFT). Para validar a abordagem proposta, primeiramente foi realizada a análise de sensibilidade dos parâmetros do algoritmo através de uma instância com 100 tarefas, determinando assim a configuração ótima dos parâmetros do BRKGA. Em seguida, a configuração otimizada foi aplicada a um conjunto de instâncias de diferentes portes (pequeno com 20 tarefas, médio com 100 tarefas e grande com 250 tarefas) para avaliar a escalabilidade e o desempenho geral do método. Os experimentos demonstram que o algoritmo é robusto e capaz de encontrar, de forma consistente, soluções de alta qualidade para o problema em estudo.

Palavras-chave: Escalonamento de Tarefas. Máquinas Paralelas. Otimização Combinatória. Metaheurísticas. BRKGA. *Makespan*.

ABSTRACT

Solving the task sequencing problem in industrial environments is a notoriously complex combinatorial optimization problem, classified as NP-hard. Essentially, the task sequencing problem, also known as job scheduling, involves defining the optimal order to execute a set of activities, considering limited resources and constraints, in order to optimize a given objective. This work proposes and evaluates a solution based on the Biased Random-Key Genetic Algorithm (BRKGA) to solve the task sequencing problem on identical parallel machines with precedence constraints, aiming to minimize the total production time (makespan). The use of a customized decoder distinguishes the developed methodology. This component translates the chromosome's random keys into a task priority list, efficiently allocating them through a constructive heuristic based on the Earliest Finish Time (EFT) criterion. To validate the proposed approach, a sensitivity analysis of the algorithm's parameters was first carried out using an instance with 100 tasks, thus determining the optimal configuration of the BRKGA parameters. Next, the optimized configuration was applied to a set of instances of different sizes (small with 20 tasks, medium with 100 tasks, and large with 250 tasks) to assess the scalability and overall performance of the method. The experiments demonstrate that the algorithm is robust and consistently capable of finding high-quality solutions for the problem under study.

Keywords: Task Scheduling. Parallel Machines. Combinatorial Optimization. Metaheuristics. BRKGA. Makespan.

LISTA DE FIGURAS

1 Exemplo de escalonamento de máquinas e cálculo do *makespan*. 4
2 Fluxograma do framework de um *Biased Random-Key Genetic Algorithm* (BRKGA). 17
3 Arquitetura geral da aplicação do BRKGA ao problema de escalonamento. 18
4 Grafo de precedência para o exemplo ilustrativo. 19
5 Exemplo de um cromossomo BRKGA para 5 tarefas. 20
6 Fluxo do processo de decodificação e avaliação de um indivíduo. 21
7 Cronograma gerado pelo decodificador para o exemplo da Figura 1. 22
8 *Boxplot* comparativo do *makespan* para as 5 melhores configurações de parâmetros. . 25
9 Tempo médio de execução do BRKGA conforme o tamanho da instância. 27

LISTA DE TABELAS

1 Análise de meta-heurísticas e outras abordagens para problemas de sequenciamento. 10
2 Tempos de processamento (p_j) das tarefas para o exemplo ilustrativo. 19
3 Parâmetros e seus respectivos valores considerados para a análise de sensibilidade. . 24
4 Parâmetros adicionais de implementação do BRKGA e seus valores utilizados. . . . 24
5 As 10 melhores configurações de parâmetros encontradas. 25
6 Resultados da análise de estabilidade e escalabilidade para as diferentes instâncias. . 26

Lista de Siglas

BRKGA	-	Biased Random-Key Genetic Algorithm
PET	-	Problema de Escalonamento de Tarefas
NEH	-	Nawaz-Enscore-Ham
SHA	-	Spotted Hyena Algorithm
SSO	-	Social Spider Optimization
CSO	-	Cuckoo Search
BAT	-	Bat Algorithm
NSGA-II	-	Non-dominated Sorting Genetic Algorithm II
LPT	-	Longest Processing Time
IG	-	Iterated Greedy
EFT	-	Earliest Finish Time

SUMÁRIO

1 Introdução **1**

2 Apresentação do Problema **3**
2.1 Processo Produtivo e Escalonamento 5
2.2 Complexidade e Objetivos de Otimização 5
2.3 Escalonamento em Máquinas Paralelas Idênticas 6

3 Trabalhos Relacionados **7**
3.1 Artigos de Revisão da Literatura 7
3.2 Modelagem Operacional 8
3.3 Meta-heurísticas para Sequenciamento de Tarefas 9
3.4 Comparação com a Literatura e Justificativa da Abordagem 10

4 Metodologia da Solução **12**
4.1 Algoritmos Evolutivos 12
4.1.1 Conceito de Codificação em Algoritmos Evolutivos 12
4.1.2 Inicialização e Avaliação 13
4.1.3 Seleção de Pais 13
4.1.4 Recombinação (Crossover) 13
4.1.5 Mutação 14
4.1.6 Seleção de Sobreviventes e Elitismo 14
4.1.7 Estrutura Genérica de um Algoritmo Evolutivo 15
4.1.8 Exemplos de Aplicação na Literatura 15
4.2 O Algoritmo BRKGA 15
4.2.1 Características Distintivas do BRKGA 16
4.2.2 O Processo Evolutivo e seus Parâmetros 16
4.3 Aplicação do BRKGA ao Problema de Sequenciamento 17
4.3.1 Exemplo Ilustrativo 18
4.3.2 Codificação da Solução no BRKGA 19
4.3.3 Decodificação e Função de Aptidão 20
4.3.4 Geração da População e Operadores Evolutivos 22

5 Resultados **23**
5.1 Configuração Experimental 23
5.1.1 Instância para Calibração 23
5.1.2 Parâmetros Testados 23

5.2 Otimização de Parâmetros 24
5.2.1 Desempenho das Melhores Configurações 24
5.2.2 Análise da Estabilidade dos Resultados 26

6 Considerações Finais **29**
6.1 Contribuições do Trabalho: 29
6.2 Limitações do Trabalho 30
6.3 Perspectivas para Trabalhos Futuros 30

1 Introdução

Com o avanço da Indústria 4.0, a busca pela eficiência na produção e a otimização do uso de recursos sempre foi um fator importante para o sucesso das operações em manufaturas. Inicialmente, com métodos artesanais e processos manuais, os desafios relacionados à organização e à distribuição de tarefas eram solucionados usando a tentativa e erro ou cálculos por força bruta. Com o advento da era digital e o desenvolvimento de técnicas avançadas, o Problema de Escalonamento de Tarefas (PET), também conhecido como programação de tarefas, passou a ser abordado por meio de métodos de otimização que buscam reduzir o tempo total de processamento e maximizar o aproveitamento dos recursos disponíveis.

Nas últimas décadas, a evolução dos métodos de pesquisa operacional e o surgimento das meta-heurísticas transformaram consideravelmente o cenário industrial. Diversos trabalhos foram desenvolvidos (1, 2), os quais mostram como técnicas de otimização podem ser aplicadas para resolver problemas complexos de sequenciamento de tarefas, apresentando alternativas eficazes quando as abordagens exatas se mostram inviáveis devido à alta complexidade computacional. É importante também dizer que, estudos focados em métodos exatos, tais como apresentado por Tanaka e Araki(3), reforçam a validade desses modelos para a obtenção de referências teóricas e comparativas de desempenho.

Neste contexto, o presente trabalho tem como objetivo desenvolver uma solução baseada no algoritmo proposto por Resende e Gonçalves(4) denominado *Biased Random-Key Genetic Algorithm* (BRKGA), aplicado ao problema de escalonamento de tarefas. O foco do problema reside na redução do tempo total de execução (*makespan*), ou seja, na minimização do tempo máximo necessário para concluir todas as tarefas, em um ambiente de máquinas paralelas idênticas com restrições de precedência.

A definição do escopo e das decisões de modelagem também foi influenciada por estudos que abordam aspectos semelhantes ao contexto deste trabalho. As ideias relacionadas à presença de máquinas paralelas, restrições de disponibilidade e janelas de tempo, bem como a estrutura de precedência entre tarefas aqui representadas por famílias, foram inspiradas no trabalho de Agárdi e Nehéz(5). A ênfase na minimização do *makespan* em ambientes com máquinas paralelas idênticas segue os princípios discutidos por Ghalami e Grosu(6). Já as ideias relacionadas à modelagem de famílias de tarefas em ambientes com máquinas paralelas foram reforçadas pelas discussões apresentadas por Fidelis e Arroyo(7).

Embora existam diversas abordagens propostas na literatura para enfrentar desafios relacionados ao PET, a robustez e a adaptabilidade do BRKGA o identificam como uma alternativa promissora como método de solução para o PET. Este trabalho, propõe integrar os avanços teóricos e as práticas consolidadas na área para desenvolver uma abordagem de solução que seja, ao mesmo tempo, eficiente e aplicável em contextos reais de produção.

O restante deste documento está dividido em mais 5 capítulos. O Capítulo 2 apresenta o con-

texto industrial atual e os desafios enfrentados no agendamento de tarefas. Em seguida, o Capítulo 3 contextualiza os trabalhos relacionados ao tema. O Capítulo 4 descreve em detalhes a abordagem proposta, com ênfase na implementação do algoritmo BRKGA e suas adaptações para o problema estudado. O Capítulo 5 apresenta e discute os resultados dos experimentos computacionais. Por fim, o Capítulo 6 traz as conclusões e perspectivas para trabalhos futuros.

2 Apresentação do Problema

O problema de escalonamento de tarefas em paralelo pode ser categorizado em três variantes principais: máquinas idênticas (P), máquinas uniformes (Q) e máquinas não relacionadas (R) (8). Neste trabalho, é examinado uma variante do problema de escalonamento de tarefas que considera máquinas idênticas com restrições de precedência para as tarefas, e tem como critério de otimização a minimização do *makespan*. Tal variação do PET foi escolhida por refletir cenários comuns em centros produtivos que operam com máquinas idênticas e buscam eficiência global (9). O escalonamento de tarefas em máquinas paralelas idênticas é um problema clássico e de grande relevância em manufaturas modernas (6, 7). Dado um conjunto de n tarefas e m máquinas iguais, o objetivo é alocar as tarefas de forma a reduzir o tempo total necessário para concluir todas as n tarefas. Esse tempo total define o cálculo do *makespan* (6).

No contexto do sequenciamento de tarefas, diferentes métricas de desempenho podem ser adotadas conforme o objetivo da produção. Duas das mais utilizadas são o *makespan* e o atraso total.

O *makespan* corresponde ao tempo total necessário para concluir todas as tarefas em um sistema, sendo igual ao intervalo entre o início da primeira tarefa e a finalização da última tarefa processada. Seu uso é comum quando se busca maximizar a eficiência global da produção, reduzindo o tempo de ocupação dos recursos. Já o atraso total considera a soma dos atrasos individuais de cada tarefa em relação ao seu prazo de entrega. Essa métrica é mais apropriada em cenários onde o cumprimento de prazos é crucial, como em ambientes sob demanda ou com penalidades contratuais (9).

Diversas variações do problema de escalonamento de tarefas em máquinas paralelas têm sido exploradas na literatura, a depender das características do ambiente produtivo. Uma dessas variações considera a presença de janelas de tempo, nas quais cada tarefa só pode ser executada dentro de um intervalo específico de disponibilidade, além de restrições de disponibilidade das máquinas. Esse tipo de modelagem do PET é abordado, por exemplo, por Agárdi e Nehéz(5), onde também são consideradas dependências entre tarefas e tempos de preparação (*setup*) específicos por máquina. Outra variação importante trata do ambiente com máquinas paralelas idênticas e busca exclusivamente a minimização do *makespan*, como apresentado por Ghalami e Grosu(6), problema que se alinha diretamente ao foco deste trabalho. Por fim, modelos que envolvem o sequenciamento de famílias de tarefas, ou seja, grupos de operações com características semelhantes, também representam uma extensão relevante do problema clássico, especialmente em sistemas produtivos com lotes ou agrupamentos de ordens, conforme discutido por Fidelis e Arroyo(7).

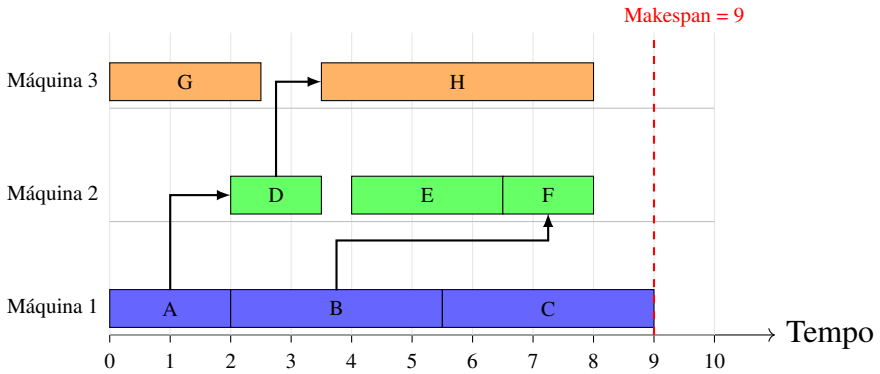
Neste trabalho, aborda-se uma variação do problema de escalonamento em máquinas paralelas idênticas que incorpora características práticas observadas em ambientes industriais. O modelo adotado considera múltiplas máquinas trabalhando em paralelo, onde cada tarefa possui um tempo de processamento previamente definido e pode depender da finalização de outras tarefas, estabelecendo assim restrições de precedência. O objetivo principal é minimizar o *makespan*, respeitando tanto as dependências entre elas quanto a disponibilidade dos recursos. Essa configuração busca representar de

forma mais realista a dinâmica de linhas de produção que operam com ordens de produção encadeadas e requisitos específicos de execução.

A classificação de problemas de escalonamento é comumente realizada através da notação clássica de três campos $(\alpha|\beta|\gamma)$, onde α define o ambiente de máquinas, β representa as restrições e características das tarefas, e γ a função objetivo. Sendo assim, a variante abordada neste trabalho é representada por $(P|prec|C_{\max})$. Nessa notação, P indica que o ambiente conta com máquinas paralelas idênticas, *prec* representa a existência de restrições de precedência entre as tarefas, e $C_{\max}$ define como critério de otimização a minimização do tempo de conclusão de todas as tarefas, conhecido como *makespan*. Esta variante é comprovadamente NP-difícil, pois pode ser reduzida do problema da partição, que consiste em dividir um conjunto de números em dois subconjuntos com somas iguais (9). Assim, abordagens exatas tornam-se inviáveis para instâncias reais de porte industrial, motivando o uso de heurísticas e meta-heurísticas capazes de fornecer boas soluções em tempos computacionais aceitáveis (4, 2).

A Figura 1 ilustra um exemplo de escalonamento em um ambiente com três máquinas paralelas idênticas e oito tarefas com durações previamente definidas. Cada linha representa uma máquina, e cada bloco representa uma tarefa, identificada por uma letra. As cores distinguem em qual máquina a tarefa foi alocada, e o comprimento de um bloco indica a duração da tarefa que ele representa. O eixo X representa o tempo e o eixo Y indica a máquina correspondente. Neste exemplo, as tarefas A, B, C, E e G são independentes, enquanto que D depende de A, F depende de B e H depende de D. O *makespan*, destacado em vermelho, corresponde ao tempo de término da última tarefa concluída (C), totalizando 9 unidades de tempo.

Figura 1: Exemplo de escalonamento de máquinas e cálculo do *makespan*.



Fonte: O autor (2025).

2.1 Processo Produtivo e Escalonamento

O processo produtivo em ambientes industriais consiste em um conjunto de operações coordenadas que transformam matérias-primas em produtos acabados. Essas operações são distribuídas entre diversos recursos, como máquinas, operadores ou linhas de montagem, que devem ser organizados para atender às demandas de produção de forma eficiente. Em geral, as ordens de produção chegam em intervalos regulares ou sob demanda, com volumes e prazos distintos, exigindo que o sistema produtivo se adapte dinamicamente às condições de operação. Além disso, certas atividades podem ter dependências entre si, exigindo que sejam executadas em uma sequência específica respeitando tais precedências, e há também variações no tempo de duração das tarefas, nos tempos de preparação e na disponibilidade dos recursos.

Dentro desse contexto, o escalonamento de tarefas é o processo de definir o instante em que cada tarefa será realizada, bem como o recurso único ao qual ela será alocada para execução. Essa decisão deve levar em conta a capacidade das máquinas, as restrições operacionais e, frequentemente, requisitos logísticos ou contratuais. Um bom escalonamento busca distribuir a carga de trabalho de forma balanceada entre os recursos, evitar períodos de ociosidade ou sobrecarga e garantir que as tarefas sejam finalizadas dentro dos prazos estabelecidos (8). Em muitos casos, é necessário considerar precedências entre operações, restrições de tempo (como janelas de liberação ou prazos de entrega), além da heterogeneidade entre as tarefas e os recursos disponíveis.

Em ambientes industriais reais, as ordens de produção são geradas continuamente e devem ser alocadas às máquinas de maneira a minimizar o *makespan*, assegurando a entrega dentro dos prazos estabelecidos e otimizando o uso dos recursos disponíveis (9). Em muitos casos, tarefas apresentam janelas de tempo específicas, períodos em que podem ser iniciadas ou finalizadas, o que adiciona complexidade ao problema (5).

2.2 Complexidade e Objetivos de Otimização

Mesmo a variante básica do problema de escalonamento de tarefas $(P||C_{\max})$ é um problema NP-difícil, pois pode ser reduzido do problema da partição (9). Diante disso, abordagens exatas se tornam inviáveis do ponto de vista computacional para instâncias de grande porte que representam de forma mais realista grandes indústrias. Isso motiva o uso de heurísticas e meta-heurísticas, que visam encontrar boas soluções em tempos computacionalmente aceitáveis (4, 2).

Heurísticas são estratégias de solução que, mesmo sem assegurar a solução ótima, conseguem encontrar resultados satisfatórios dentro de um tempo computacional viável. Em contraste, os métodos exatos encontram a solução ótima quando conseguem resolver o problema. No entanto, se o problema envolver muitas combinações e essas não forem devidamente gerenciadas, o tempo necessário para encontrar a resposta pode crescer de forma exponencial.

2.3 Escalonamento em Máquinas Paralelas Idênticas

O escalonamento em máquinas paralelas idênticas é amplamente utilizado em ambientes industriais onde tarefas devem ser distribuídas entre máquinas com a mesma capacidade, visando a minimização do *makespan* (9, 6). Apesar da simplicidade na definição do problema, essa classe pode incorporar diversas restrições práticas, como:

- **Janelas de tempo:** intervalos específicos durante os quais uma determinada tarefa está autorizada a iniciar e terminar sua execução.
- **Disponibilidade de recursos:** limitação de tempo ou capacidade em que máquinas ou outros recursos produtivos estão aptos a operar.
- **Precedência entre tarefas:** dependências operacionais que indicam que uma tarefa só pode ser iniciada após a conclusão de outra, como ocorre em famílias de ordens de produção encadeadas.

A presença de tais restrições práticas eleva a complexidade do problema, tornando o uso de métodos aproximados, como heurísticas e meta-heurísticas, uma abordagem viável para encontrar soluções de alta qualidade em tempo hábil (2, 4). Diante desse cenário, este trabalho foca especificamente no problema de escalonamento de tarefas em máquinas paralelas idênticas com restrições de precedência e tempos de processamento fixos, sem considerar janelas de tempo ou tempos de preparação (*setup*). O objetivo é minimizar o *makespan*, promovendo uma alocação eficiente das tarefas entre os recursos disponíveis. Essa configuração reflete cenários industriais comuns e permite avaliar o desempenho do algoritmo proposto em um ambiente controlado, porém representativo.

3 Trabalhos Relacionados

Na literatura, diversas abordagens têm sido propostas para resolver o problema de sequenciamento de famílias de tarefas em máquinas paralelas idênticas (7). Esses trabalhos variam tanto em suas técnicas de solução quanto em suas aplicações. Muitos utilizam métodos de otimização clássicos, como programação inteira e heurísticas, enquanto outros exploram meta-heurísticas, como algoritmos genéticos (5) e outras estratégias bio-inspiradas, como algoritmo da hiena-malhada (2), no intuito de melhorar o desempenho do escalonamento.

A pesquisa por trabalhos presentes na literatura foi feita com base em palavras-chave relacionadas ao sequenciamento de tarefas em ambientes de máquinas paralelas, utilizando ferramentas como Google Acadêmico[1] e Research Rabbit[2]. Foram aplicadas técnicas de *snowballing* para mapear tanto os trabalhos mais recentes (10, 6, 9), incluindo artigos citados e que citam os documentos primeiramente encontrados e analisados. Quanto aos estudos seminais que deram origem a essa linha de pesquisa, pode-se referenciar o trabalho proposto por Tanaka e Araki(3).

A seguir, a Subseção 3.1 apresenta artigos de revisão da literatura, enquanto as Subseções 3.2 e 3.3 discutem os trabalhos voltados para modelos de programação matemática e meta-heurísticas, respectivamente.

3.1 Artigos de Revisão da Literatura

O trabalho de Laha e Behera(11) apresenta uma revisão e avaliação de quatro algoritmos construtivos populares aplicados ao problema de sequenciamento de tarefas independentes em máquinas paralelas idênticas, visando a minimização do *makespan* $(P||C_{max})$. Os algoritmos analisados, todos discutidos em profundidade pelos autores, são:

- ***Longest Processing Time*** **(LPT):** Considerado o algoritmo mais simples, ordena as tarefas em ordem decrescente de seu tempo de processamento e as aloca, uma a uma, na máquina que se tornará livre mais cedo.

- **MULTIFIT:** Aborda o problema como uma variação de empacotamento de caixas (*bin packing*), utilizando busca binária para encontrar a menor "capacidade" (ou seja, o menor *makespan*) na qual todas as tarefas podem ser alocadas às máquinas.

- **COMBINE:** Um método híbrido que primeiro executa o LPT para obter uma boa solução inicial e, em seguida, usa o resultado como um limite superior para refinar a busca do algoritmo MULTIFIT.

[1] scholar.google.com.br
[2] researchrabbitapp.com

- **LISTFIT:** Um algoritmo mais elaborado que combina os princípios do LPT e do MULTIFIT, mas que opera com múltiplas listas de tarefas, criadas e atualizadas ao longo de várias iterações para produzir uma solução final de alta qualidade.

O estudo combina uma análise crítica da literatura desses métodos com uma avaliação experimental comparativa de seu desempenho. Os autores concluem que, embora o LPT seja computacionalmente mais eficiente, o LISTFIT geralmente apresenta melhor qualidade de solução na maioria das instâncias, ainda que exija maior tempo de processamento. Esta revisão é particularmente relevante por seu foco em heurísticas construtivas, que frequentemente servem de base para o desenvolvimento de meta-heurísticas mais sofisticadas.

No âmbito das meta-heurísticas, e sendo o BRKGA o método central deste trabalho, a revisão da literatura apresentada por Resende et al.(12) fornece um panorama extensivo sobre o assunto. Este estudo examina mais de 150 publicações, detalhando a aplicação do BRKGA em uma vasta gama de problemas de otimização combinatória. Dentre estes, os problemas de sequenciamento são identificados como a categoria de aplicação mais prevalente no uso do BRKGA. Adicionalmente, a análise de Resende et al.(12) aponta ainda tendências chave no BRKGA, como sua hibridização usual com algoritmos de busca local para aprimorar as soluções e a adição de funcionalidades para promover maior diversidade populacional, o que é crucial para evitar a convergência prematura. Esta revisão acaba sendo fundamental para contextualizar o estudo desse trabalho.

Em (13), os autores apresentam um algoritmo iterativo baseado em estratégias de destruição e reconstrução para o sequenciamento de tarefas no problema *flowshop*, no qual todas as tarefas devem ser processadas pelas máquinas na mesma ordem. O método proposto combina simplicidade de implementação com eficácia, e utiliza a heurística NEH (*Nawaz–Enscore–Ham*) (14) na fase de reconstrução e, opcionalmente, busca local para refinamento da solução. Os resultados experimentais demonstram que o algoritmo é competitivo com técnicas mais complexas, destacando-se como uma alternativa eficiente e de fácil aplicação. Para validar sua abordagem, os autores realizaram testes utilizando o conhecido conjunto de instâncias de *benchmark* proposto por Taillard(15), que engloba problemas com um número de tarefas variando de 20 a 500 e um número de máquinas entre 5 e 20. É importante notar que, embora essas instâncias tenham sido originalmente propostas para o problema de *flowshop*, sua estrutura foi adaptada neste TCC para gerar os cenários de teste do problema de máquinas paralelas.

3.2 Modelagem Operacional

Trabalhos que apresentam modelos de programação matemática exploram técnicas exatas e heurísticas para o sequenciamento de tarefas. Por exemplo, Tanaka e Araki(3) usam um algoritmo *branch and bound* com relaxação Lagrangiana para minimizar o atraso total em máquinas paralelas idênticas. O método utiliza condições de dominância para restringir a árvore de busca e aprimorar os limites inferiores. Os experimentos computacionais demonstraram que o algoritmo resolveu de forma

ótima instâncias com até 25 tarefas. Os autores verificaram que, para um número fixo de tarefas, o problema se torna computacionalmente mais simples à medida que o número de máquinas (testado de 2 a 10) aumenta. Consequentemente, as limitações de escalabilidade do método estão associadas principalmente ao número de tarefas, e não ao de máquinas, tornando mais eficaz para problemas de pequeno e médio porte neste ponto.

No trabalho publicado por Tanaka e Araki(3), os autores apresentam um avanço em relação a abordagens exatas anteriores para o mesmo problema, ao introduzirem o uso de relaxação Lagrangiana no cálculo de limites inferiores, em conjunto com condições de dominância para poda da árvore de busca. Essa combinação permitiu resolver de forma ótima instâncias significativamente maiores, com até 25 tarefas e qualquer número de máquinas, superando a escalabilidade alcançada por trabalhos anteriores. Previamente, Azizoğlu e Kirca(16) também propuseram um algoritmo *branch and bound*, que conseguia resolver otimamente instâncias com até 12 tarefas e 3 máquinas. Posteriormente, Yalaoui e Chu(17) aprimoraram este método com a adição de novas condições de dominância, elevando a capacidade de solução para instâncias com até 15 tarefas. Em outra frente de pesquisa relevante, Shim e Kim(18) desenvolveram um algoritmo *branch and bound* com limites inferiores mais eficazes, superando os propostos por Azizoğlu e Kirca(16). A evolução dessas técnicas evidencia um esforço contínuo na literatura para expandir a fronteira dos problemas solucionáveis de forma ótima, onde o principal desafio reside em conter a explosão combinatória com o aumento do número de tarefas.

3.3 Meta-heurísticas para Sequenciamento de Tarefas

O uso de meta-heurísticas tem se consolidado como uma abordagem eficaz para resolver problemas de sequenciamento de tarefas, especialmente em cenários onde métodos exatos enfrentam limitações devido à complexidade computacional e falham na escalabilidade. Um exemplo relevante é o trabalho de Mzili et al.(2), que apresenta um algoritmo de otimização baseado no comportamento de hienas (*Spotted Hyena Algorithm* - SHA). O método é aplicado ao problema de *flowshop* e demonstra resultados competitivos, superando diversas outras técnicas bio-inspiradas como *Social Spider Optimization* (SSO), *Cuckoo Search* (CSO) e *Bat Algorithm* (BAT) em termos de tempo de execução e qualidade das soluções nos testes apresentados.

Outro exemplo é apresentado por Zhao e Yuan(1), onde o algoritmo genético multiobjetivo *Non-dominated Sorting Genetic Algorithm II* (NSGA-II) é utilizado para integrar manutenção preditiva e sequenciamento de produção. O estudo otimiza simultaneamente critérios de custo e eficiência operacional, evidenciando o potencial de algoritmos evolutivos para problemas complexos.

Rubén e Stützle(13) propõem uma meta-heurística baseada em busca tabu para o sequenciamento de tarefas de famílias em máquinas paralelas idênticas de processamento em lote. A pesquisa demonstra que a técnica pode gerar soluções de alta qualidade para problemas com restrições adicionais, como tempos de configuração dependentes da máquina e do trabalho.

Por fim, Xavier, Lins e Silva(19) aborda o problema de sequenciamento de máquinas paralelas

idênticas com o objetivo de minimizar o *makespan*. O estudo apresenta um algoritmo de aproximação paralelo que, embora não garanta a solução ótima, consegue equilibrar eficiência computacional e qualidade das soluções.

A Tabela 1 resume os principais trabalhos citados nesta subseção, destacando as técnicas empregadas, os resultados obtidos e as bases de dados utilizadas para validação.

Tabela 1: Análise de meta-heurísticas e outras abordagens para problemas de sequenciamento.

Referências	**Técnica Aplicada**	**Base de Dados**	**Resultados Obtidos**
Mzili et al.(2)	*Spotted Hyena Algorithm*	Instâncias *benchmark* de Carlier, Reeves e Taillard (até 75×20).	Encontrou a solução ótima para todas as instâncias de teste, superando algoritmos como CSO e BAT.
Zhao e Yuan(1)	NSGA-II (Algoritmo Genético Multiobjetivo)	Estudo de caso real com 6 máquinas e 6 trabalhos (6×6).	Obteve soluções de Pareto, otimizando simultaneamente o custo de processamento e o tempo de produção.
Rubén e Stützle(13)	*Iterated Greedy* (IG) para *Flowshop* com Permutação	Instâncias *benchmark* de Taillard (até 500×20).	Resultados superiores aos métodos da literatura, estabelecendo novos melhores resultados para várias instâncias.
Xavier, Lins e Silva(19)	Fluxo de Custo Mínimo em Redes	Dados reais de uma indústria de calçados (até 18 tarefas).	Redução de 30% no tempo total de produção (*makespan*) em comparação com o método da empresa.
Fonte: O autor (2025)			

3.4 Comparação com a Literatura e Justificativa da Abordagem

A literatura sobre sequenciamento de tarefas em máquinas paralelas apresenta um espectro de abordagens, desde heurísticas construtivas e métodos exatos até meta-heurísticas complexas. O trabalho de Laha e Behera(11), por exemplo, oferece um panorama sobre heurísticas construtivas como LPT e LISTFIT, baseado em princípios de empacotamento de caixas (*bin packing*) e agendamento em lista. Embora esses métodos sejam rápidos, eles não garantem a mesma qualidade de solução que abordagens mais robustas podem alcançar. No outro extremo, métodos exatos como o *branch and bound* de Tanaka e Araki(3) encontram a solução ótima, mas sua aplicabilidade é restrita a problemas de pequeno porte (até 25 tarefas, no caso dele), sendo inviáveis para cenários industriais maiores devido à explosão combinatória.

Diante desse cenário, as meta-heurísticas surgem como uma alternativa promissora, equilibrando qualidade da solução e tempo computacional. Abordagens como o *Iterated Greedy* (IG) de Rubén e Stützle(13) e outras baseadas em inteligência de enxames, como o *Spotted Hyena Algorithm* (2), demonstram grande eficácia. No entanto, para este trabalho, foi escolhido o *Biased Random-Key Genetic Algorithm* (BRKGA) (4). A justificativa para essa escolha se baseia em três pilares:

- **Flexibilidade e Reusabilidade:** O BRKGA desacopla a lógica da meta-heurística da solução do problema, que fica a cargo de um "decodificador". Essa arquitetura é particularmente poderosa, pois permite que o decodificador utilize uma heurística construtiva rápida e eficiente, como a baseada em EFT, adotada neste trabalho, para traduzir as soluções genéricas do BRKGA em escalonamentos viáveis e de alta qualidade.
- **Exploração Ampla:** Por ser um algoritmo populacional, o BRKGA explora múltiplas regiões do espaço de busca simultaneamente, o que aumenta as chances de evitar ótimos locais em comparação com algoritmos de trajetória única, como o IG.
- **Eficácia Comprovada:** Conforme apontado na revisão de Resende et al.(12), o BRKGA tem um vasto histórico de sucesso na resolução de problemas de sequenciamento, sendo esta sua área de aplicação mais prevalente e bem-sucedida.

A literatura revisada classifica as técnicas de solução para o sequenciamento de tarefas em métodos exatos, heurísticas e meta-heurísticas. Este trabalho foca no uso de meta-heurísticas, que oferecem um bom equilíbrio entre a qualidade da solução e o tempo de processamento para problemas complexos. Diferentemente dos métodos exatos, que possuem limitações de escalabilidade para problemas de grande porte (3), a abordagem meta-heurística se mostra especialmente adequada aos desafios de cenários industriais quando utilizada para guiar heurísticas construtivas na busca por soluções de alta qualidade. Dentre as meta-heurísticas, este trabalho propõe a aplicação do *Biased Random-Key Genetic Algorithm* para guiar a heurística construtiva EFT (*Earliest Finish Time*) na resolução do problema $P|prec|C_{max}$, cuja metodologia e justificativas foram detalhadas na Subseção 3.4.

4 Metodologia da Solução

Este capítulo detalha a metodologia de solução proposta para o problema de sequenciamento de tarefas. A seção inicial apresenta o contexto dos Algoritmos Evolutivos (Seção 4.1), que fundamentam a abordagem. Em seguida, é descrito o BRKGA, o método específico utilizado neste trabalho (Seção 4.2). Por fim, a Seção 4.3 detalha os aspectos práticos da implementação, abordando desde a configuração dos parâmetros do algoritmo e a lógica de codificação e decodificação, até a modelagem das restrições de precedência necessárias para a construção de um escalonamento de produção válido.

4.1 Algoritmos Evolutivos

Algoritmos Evolutivos (AEs) constituem uma classe de meta-heurísticas inspiradas nos mecanismos da evolução biológica, como seleção natural, reprodução e mutação. Eles operam sobre uma população de soluções candidatas que evoluem ao longo de várias gerações, guiadas por um processo de avaliação baseado em uma função de aptidão. Devido à sua flexibilidade e capacidade de explorar grandes espaços de busca de forma eficiente, os AEs têm sido amplamente utilizados na resolução de problemas complexos de otimização combinatória, incluindo problemas de sequenciamento de tarefas.

Nesta seção, os conceitos fundamentais dos Algoritmos Evolutivos (AEs) são apresentados. A discussão a seguir, incluindo a descrição das etapas do ciclo evolutivo e os exemplos de operadores, é baseada integralmente na obra de referência de Eiben e Smith(20). Sempre que pertinente, serão feitas analogias diretas com a aplicação prática desenvolvida neste trabalho.

4.1.1 Conceito de Codificação em Algoritmos Evolutivos

A codificação da solução, também chamada de representação genotípica, é uma das etapas mais importantes em um Algoritmo Evolutivo, pois define como cada solução do problema será expressa dentro do algoritmo, ou seja, como o espaço de busca é estruturado. Cada solução é representada por um cromossomo, que é uma estrutura de dados composta por um conjunto de alelos, geralmente armazenados como vetores.

A forma como esse cromossomo é construído depende do tipo de problema. Para problemas de otimização combinatória, como o sequenciamento de tarefas, uma representação comum é a de permutações, onde cada alelo representa uma tarefa e sua posição no vetor define a ordem de execução. Outras representações possíveis incluem vetores binários (usados em problemas de seleção), vetores de números inteiros ou reais como é o caso das representações por chaves aleatórias, entre outras.

A escolha adequada da codificação influencia diretamente na qualidade dos operadores de cruzamento e mutação, bem como na capacidade do algoritmo de gerar soluções viáveis (20). Portanto, é essencial que a codificação seja compatível com as características do problema e com as estratégias de decodificação e avaliação adotadas.

No contexto deste trabalho, utilizando o BRKGA, a codificação da solução é realizada naturalmente por chaves aleatórias. Cada cromossomo é um vetor de números reais no intervalo $[0,1)$, que serve como entrada para o decodificador, responsável por traduzir essas chaves em uma solução viável para o problema específico.

4.1.2 Inicialização e Avaliação

O algoritmo se inicia com o procedimento de **Inicialização**, no qual uma população de indivíduos é criada, geralmente de forma aleatória para garantir a diversidade inicial desta população (soluções). Cada indivíduo, que representa uma solução para o problema, passa então pela **Avaliação**. Nesta etapa, uma **função de aptidão (*fitness function*)** calcula a qualidade de cada solução de acordo com o objetivo do problema em estudo. No contexto deste trabalho, o *fitness* de um indivíduo corresponde ao valor do *makespan* (C_{max}) do cronograma representado pelo seu cromossomo, onde valores menores indicam indivíduos mais aptos, isto é, com um tempo menor de execução das tarefas.

4.1.3 Seleção de Pais

A **Seleção** é o processo que escolhe os indivíduos da população atual (pais) para criar os descendentes da próxima geração. O princípio fundamental é o da "sobrevivência do mais apto", onde indivíduos com maior *fitness* têm uma probabilidade maior de serem selecionados. Dentre as várias técnicas existentes, destacam-se (20):

- **Seleção Proporcional à Aptidão (*Fitness-Proportionate Selection*):** Frequentemente implementada como Seleção por Roleta, onde a probabilidade de um indivíduo ser escolhido é proporcional ao seu valor de *fitness* em relação à média da população.
- **Seleção por Torneio (*Tournament Selection*):** Um pequeno grupo de indivíduos é selecionado aleatoriamente da população, e o melhor indivíduo deste grupo é escolhido como um dos pais. É um dos métodos mais utilizados na prática.

Neste trabalho, a seleção de pais segue a lógica do elitismo proposta pelo BRKGA, que será detalhada na Seção 4.2. Nela, a população é dividida em dois grupos com base no seu valor de *fitness*: um conjunto de elite, que contém as melhores soluções encontradas, e um conjunto de não-elite. Os descendentes são então gerados a partir do cruzamento entre um pai de elite e um pai não-elite.

4.1.4 Recombinação (Crossover)

A **Recombinação**, ou *crossover*, cria um ou mais descendentes (filhos) a partir dos pais selecionados na etapa anterior. É o principal operador responsável pela **intensificação**, ou seja, a exploração de regiões promissoras do espaço de busca. Como este trabalho lida com problemas de sequenciamento, a representação é baseada em permutações, e operadores específicos são necessários para garantir que os descendentes sejam permutações válidas, como:

- **Crossover de Ordem (OX):** Um dos operadores mais conhecidos para permutações. Ele copia uma subsequência aleatória de um dos pais e, em seguida, preenche os genes restantes com o material do outro pai, mantendo a ordem relativa em que eles aparecem.
- **Crossover Parcialmente Mapeado (PMX):** Também muito utilizado, este operador troca uma subsequência entre os dois pais e, em seguida, utiliza uma regra de mapeamento para resolver conflitos de genes duplicados, preservando uma combinação de ordem e valor absolutos dos pais.

Neste trabalho, empregamos a técnica de recombinação uniforme parametrizado, onde os alelos dos descendentes são herdados dos pais de elite ou não-elite com base em uma probabilidade definida (p_e).

4.1.5 Mutação

A **Mutação** introduz pequenas e aleatórias alterações nos cromossomos dos descendentes. Sua função é a **diversificação**, essencial para evitar que o algoritmo fique preso em ótimos locais e para introduzir novo material genético na população. Exemplos de operadores de mutação para sequências incluem:

- **Mutação por Troca (*Swap*):** Simplesmente seleciona duas posições aleatórias no cromossomo e troca seus valores.
- **Mutação por Inserção (*Insert*):** Retira um gene de uma posição e o reinsere em outra posição aleatória.

Neste trabalho, a mutação é implementada pela introdução de novos indivíduos, chamados "mutantes", que são gerados aleatoriamente (21). Diferente de um operador de mutação tradicional que modifica alelos de indivíduos existentes, no BRKGA, esses mutantes são criados independentemente e inseridos diretamente na nova população, sem que os filhos resultantes do processo de recombinação (*crossover*) sejam submetidos a uma fase de modificação pós-cruzamento (12).

4.1.6 Seleção de Sobreviventes e Elitismo

A etapa final do ciclo, também chamada de **Substituição**, decide quais indivíduos formarão a população da próxima geração. Uma estratégia fundamental é o **Elitismo**, que consiste em garantir que o(s) melhor(es) indivíduo(s) da geração atual seja(m) transferido(s) diretamente para a próxima, assegurando que a melhor solução encontrada nunca seja perdida.

Neste trabalho, a seleção de sobreviventes incorpora o elitismo, onde os indivíduos elite da geração atual são diretamente passados para a próxima. A próxima população é então composta por um número fixo desses indivíduos elite, um número fixo de mutantes gerados aleatoriamente, e um número fixo de descendentes resultantes do processo de recombinação (21).

4.1.7 Estrutura Genérica de um Algoritmo Evolutivo

O fluxo de um Algoritmo Evolutivo pode ser resumido pelo pseudo-código genérico apresentado no Algoritmo 1.

Algoritmo 1 AlgoritmoEvolutivo()

```
1: t ← 0
2: p(t) ← InicializarPopulação()
3: Avaliar população p(t)
4: while critério de parada não satisfeito do
5:     Pais(t) ← Seleção(p(t))
6:     Descendentes(t) ← Cruzamento(Pais(t))
7:     Descendentes'(t) ← Mutação(Descendentes(t))
8:     Avaliar Descendentes'(t)
9:     p(t+1) ← Seleção de Sobreviventes(p(t), Descendentes'(t))
10:    t ← t+1
11: end while
12: return melhor indivíduo de p(t)
```

Fonte: O autor (2025).

4.1.8 Exemplos de Aplicação na Literatura

Os componentes do ciclo evolutivo aqui descritos são aplicados de diversas formas nos trabalhos da área de sequenciamento. Por exemplo, Agárdi e Nehéz(5), ao resolverem um problema de máquinas paralelas, utilizam um Algoritmo Genético com operadores de recombinação clássicos para permutações, como o *Crossover* de Ordem (OX) e o *Crossover* Parcialmente Mapeado (PMX). Para a diversificação, empregam o operador de mutação 2-opt.

De forma similar, Schaller(9) também aplica um Algoritmo Genético para o sequenciamento em máquinas paralelas, mas com uma configuração diferente, utilizando a seleção por torneio, *crossover* de ordem uniforme e mutação por deslocamento (*shift mutation*). Já em um cenário mais complexo, Zhao e Yuan(1) utilizam uma variação especializada dos AEs, o NSGA-II, para resolver um problema multiobjetivo que integra produção e manutenção, demonstrando a adaptabilidade do *framework*.

Esses exemplos ilustram como a estrutura geral dos AEs é uma base flexível, onde a escolha e a combinação de diferentes operadores de seleção, cruzamento e mutação permitem adaptar a técnica para as especificidades de diferentes problemas de otimização.

4.2 O Algoritmo BRKGA

O *Biased Random-Key Genetic Algorithm* (BRKGA) é uma meta-heurística de busca geral para encontrar soluções ótimas ou quase ótimas para problemas complexos de otimização combinatória. Proposto por Resende e Gonçalves(4), ele se enquadra na categoria de algoritmos genéticos, mas se distingue por um conjunto de características específicas que padronizam e simplificam sua aplicação (21).

4.2.1 Características Distintivas do BRKGA

Conforme Resende e Toso(21), o BRKGA se diferencia de um Algoritmo Genético tradicional por três características principais, que resultam em um *framework* padronizado e de fácil reutilização, as quais são brevemente descritas a seguir.

- **Codificação por Chaves Aleatórias:** A representação de uma solução (cromossomo) é sempre um vetor de números reais no intervalo [0, 1), chamados de chaves aleatórias. Essa representação é agnóstica ao problema, ou seja, o motor evolutivo do algoritmo não precisa ser alterado para diferentes aplicações. A tarefa de converter esse vetor genérico em uma solução viável para o problema específico (como um cronograma) fica a cargo de um algoritmo chamado **decodificador**, que é a única parte dependente do problema a ser implementada pelo usuário.

- **Recombinação Enviesada (*Biased Crossover*):** O processo de cruzamento para gerar descendentes é bem definido e não requer operadores complexos. Para cada gene do novo indivíduo, uma moeda enviesada é lançada: com uma alta probabilidade ρ_e, o gene é herdado do pai mais apto (do conjunto de elite). Caso contrário, é herdado do outro pai (do conjunto não elite). Esse viés em favor da elite acelera a convergência para boas soluções.

- **Introdução de Mutantes:** Em vez de um operador de mutação que altera levemente os cromossomos existentes, o BRKGA introduz diversidade na população através da criação de "mutantes". A cada nova geração, uma pequena porção da população é preenchida com indivíduos completamente novos, gerados de forma aleatória da mesma maneira que a população inicial.

4.2.2 O Processo Evolutivo e seus Parâmetros

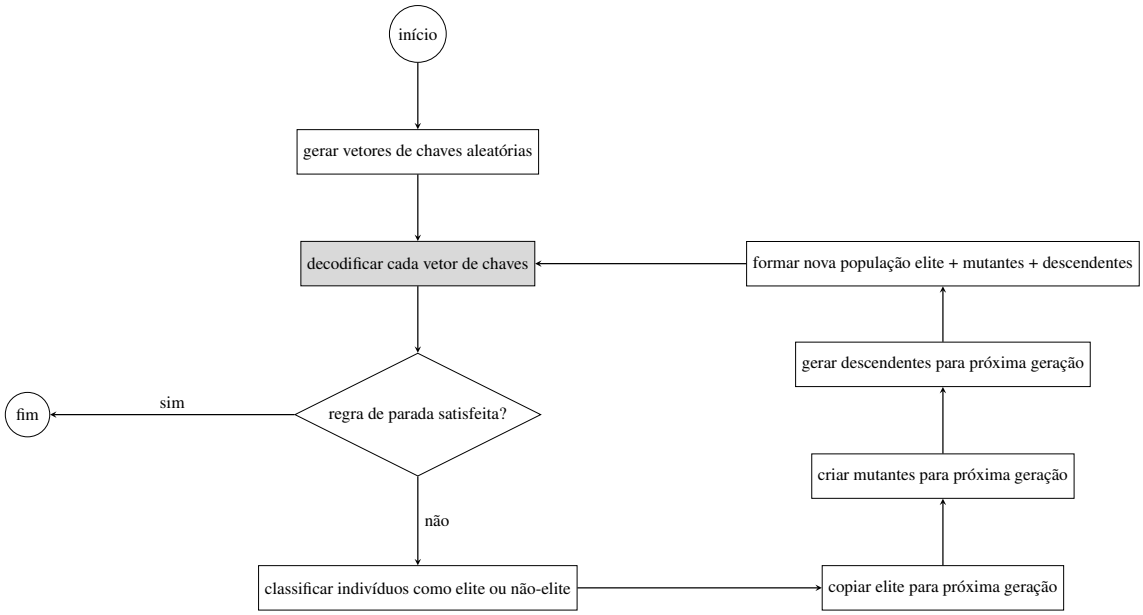
O fluxo evolutivo do BRKGA, ilustrado no fluxograma da Figura 2, ocorre ao longo de várias gerações. Em cada geração, uma população de $|p|$ indivíduos é avaliada e particionada em dois grupos: um pequeno conjunto de $|p_e|$ indivíduos de elite (as melhores soluções) e o restante de indivíduos não elite. A população da próxima geração é formada por três grupos:

- Uma cópia exata do conjunto de elite.

- Um conjunto de $|p_m|$ indivíduos mutantes gerados aleatoriamente.

- O restante da população, preenchido com descendentes gerados pela recombinação enviesada entre um pai de elite e um pai não elite.

Figura 2: Fluxograma do framework de um *Biased Random-Key Genetic Algorithm* (BRKGA).



Fonte: Adaptado de Resende e Toso(21).

Para sua execução, o algoritmo requer a configuração de alguns parâmetros chave, conforme descrito por Resende e Toso(21):

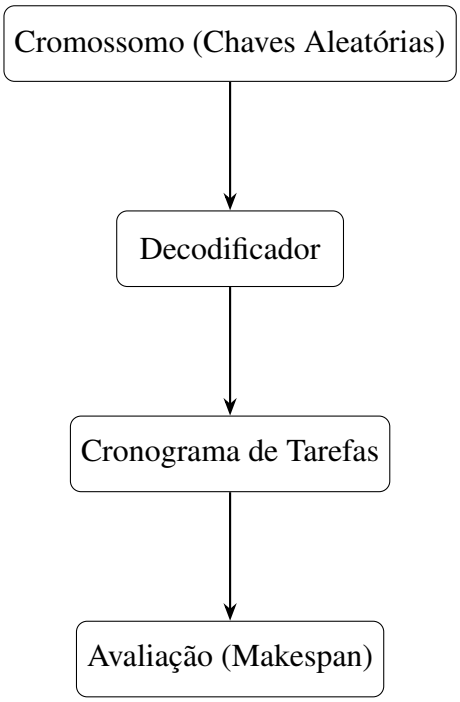
- n: O número de genes (chaves aleatórias) em cada cromossomo.
- $|p|$**:** O número total de indivíduos na população.
- $|p_e|$**:** A quantidade de indivíduos que farão parte do conjunto de elite.
- $|p_m|$**:** A quantidade de indivíduos mutantes a serem inseridos a cada nova geração.
- ρ_e**:** A probabilidade de um descendente herdar um gene do seu pai de elite.

A separação entre o *framework* geral do BRKGA e o decodificador específico do problema, que será detalhado na Seção 4.3, é o que confere ao método sua robustez e flexibilidade.

4.3 Aplicação do BRKGA ao Problema de Sequenciamento

Após a apresentação dos conceitos gerais do *framework* BRKGA, esta seção tem como objetivo detalhar como o algoritmo foi efetivamente aplicado para resolver o problema de sequenciamento de tarefas em máquinas paralelas idênticas com restrições de precedência ($P|prec|C_{max}$), que é o foco deste trabalho. Para ilustrar cada etapa do processo, desde a representação de uma solução até o cálculo de sua qualidade, será utilizado um exemplo simplificado.

Figura 3: Arquitetura geral da aplicação do BRKGA ao problema de escalonamento.



Fonte: O autor (2025).

4.3.1 Exemplo Ilustrativo

Para ilustrar o funcionamento da metodologia usada neste trabalho, considere um problema de pequeno porte com as seguintes características:

- **Máquinas (m):** 2 máquinas paralelas idênticas (M1, M2).
- **Tarefas (n):** 5 tarefas que devem ser executadas ($T1$, $T2$, $T3$, $T4$, $T5$).
- **Tempos de Processamento (p_j):** Tempo de processamento de cada tarefa j, definidos na Tabela 2. Os tempos de processamento (p_j) estão expressos em unidades de tempo arbitrárias, que podem representar segundos, minutos ou horas, conforme a aplicação.
- **Restrições de Precedência (prec):** $T1$ deve ser concluída antes de $T3$ ($T1 \rightarrow T3$); e $T2$ deve preceder $T4$ ($T2 \rightarrow T4$), enquanto a tarefa $T5$ é independente. Esta relação é mostrada no grafo da Figura 4.

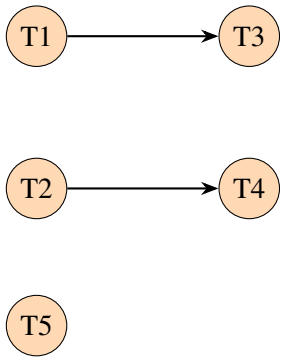
O objetivo é encontrar um cronograma de alocação das 5 tarefas nas 2 máquinas que minimize o tempo total de conclusão, o *makespan*.

Tabela 2: Tempos de processamento (p_j) das tarefas para o exemplo ilustrativo.

Tarefa	(p_j)
$T1$	$p_1 = 3$
$T2$	$p_2 = 5$
$T3$	$p_3 = 2$
$T4$	$p_4 = 4$
$T5$	$p_5 = 3$

Fonte: O autor (2025).

Figura 4: Grafo de precedência para o exemplo ilustrativo.



Fonte: O autor (2025).

4.3.2 Codificação da Solução no BRKGA

No *framework* BRKGA, a representação de uma solução é padronizada e se desvincula da estrutura do problema. Cada solução, codificada no **cromossomo** do indivíduo, é representada por um **vetor de chaves aleatórias**, que é a estrutura de dados fundamental passada ao algoritmo.

Estrutura do Cromossomo O cromossomo é um vetor de tamanho n, onde n é o número total de tarefas a serem sequenciadas. Para o nosso exemplo ilustrativo com 5 tarefas, o cromossomo terá, portanto, 5 posições (ou alelos). Cada alelo é composto por duas partes de informação:

- **A Posição (Locus):** A posição de um alelo no vetor está estaticamente associada a uma tarefa específica. Por convenção, a posição i do vetor corresponde à tarefa i (T_i). Assim, a primeira posição sempre se refere à $T1$, a segunda à $T2$, e assim por diante.

- **O Valor (Chave Aleatória):** O valor contido em cada alelo é um número real gerado aleatoriamente no intervalo [0, 1). Este valor não tem um significado físico direto (como tempo, custo ou ordem), mas serve unicamente para atribuir uma **prioridade** à tarefa associada à sua posição.

Lógica de Preenchimento e Exemplo A lógica de preenchimento do cromossomo, ou seja, a codificação de uma solução do PET para gerar um cromossomo na população inicial (e para os mutantes) é a mais simples possível: para cada uma das n posições do vetor, um novo número é sorteado de uma distribuição uniforme contínua em [0, 1). Este processo é repetido para cada indivíduo da população. A Figura 5 ilustra um cromossomo de exemplo para o nosso problema de 5 tarefas. A ordenação desses valores de chave aleatória, do menor para o maior, irá posteriormente definir a sequência de prioridade das tarefas a serem alocadas pelo decodificador.

Figura 5: Exemplo de um cromossomo BRKGA para 5 tarefas.

Posição (*Locus*)	1	2	3	4	5
Tarefa Associada	$T1$	$T2$	$T3$	$T4$	$T5$
Chave Aleatória (Valor)	0.81	0.23	0.95	0.44	0.15

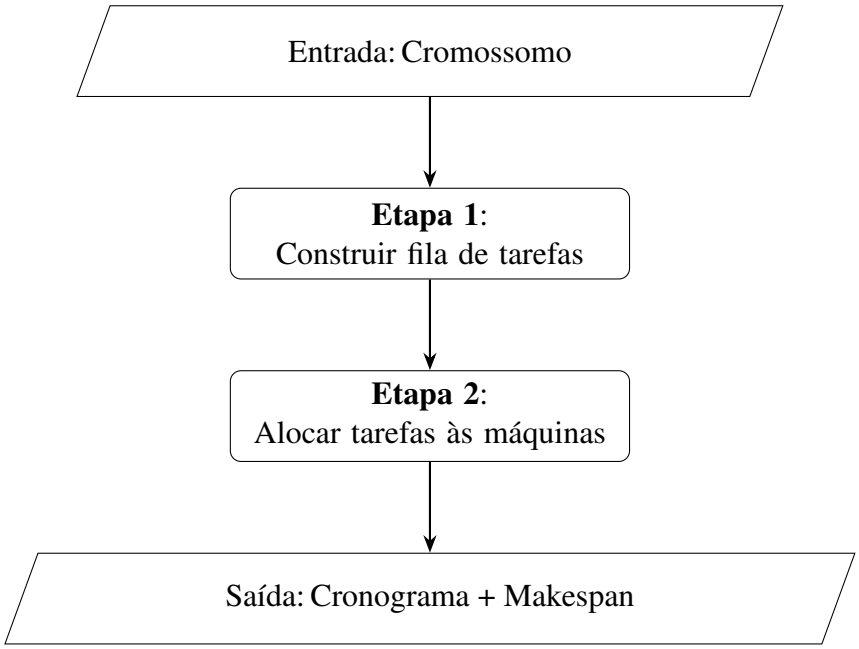
Fonte: O autor (2025)

Neste exemplo, a Tarefa 5 ($T5$), com a menor chave (0.15), receberá a maior prioridade de execução, enquanto a Tarefa 3 ($T3$), com a maior chave (0.95), terá a menor prioridade. Este cromossomo é a entrada para o algoritmo **decodificador**, que o traduzirá em um cronograma de produção viável.
Embora o cromossomo represente uma possível ordem de execução das tarefas, ele não é uma solução viável por si só. A viabilidade do escalonamento é garantida pelo decodificador, que interpreta as prioridades definidas pelas chaves aleatórias, respeitando as restrições de precedência e capacidade de recursos. Assim, o BRKGA utiliza o cromossomo como uma heurística de ordenação inicial, a partir da qual constrói uma solução viável para o problema através do decodificador proposto neste trabalho.

4.3.3 Decodificação e Função de Aptidão

O decodificador é o componente responsável por traduzir um cromossomo em uma solução viável para o problema, respeitando as restrições de precedência entre as tarefas e as capacidades das máquinas. A saída é um cronograma completo, cuja qualidade é avaliada por meio do *makespan*.
O processo de decodificação ocorre em duas etapas principais: *(i)* construção de uma fila de tarefas prontas para execução, baseada nas chaves aleatórias e no grau de entrada (*inDegree*) de cada tarefa; e *(ii)* alocação de cada tarefa na máquina com menor tempo de término viável, de forma a respeitar todas as precedências. Esse fluxo é ilustrado na Figura 6.

Figura 6: Fluxo do processo de decodificação e avaliação de um indivíduo.



Fonte: O autor (2025).

Etapa 1. Identificação das Tarefas Prontas para Execução Inicialmente, o decodificador identifica as tarefas que não possuem predecessores (grau de entrada zero). Essas tarefas são adicionadas a uma fila de prontidão, implementada como uma fila de prioridade, onde a prioridade é determinada pelas chaves aleatórias do cromossomo: quanto menor a chave, maior a prioridade da tarefa.

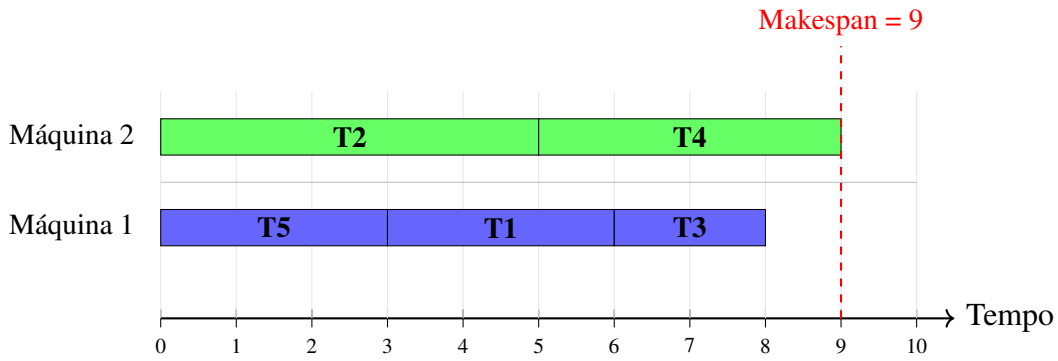
Etapa 2. Alocação Progressiva com Precedência As tarefas da fila de prontidão são avaliadas uma a uma, seguindo a ordem de prioridade. Para cada tarefa, o algoritmo primeiro determina seu **tempo de liberação**, que é o instante de tempo em que a última de suas tarefas predecessoras é concluída. Uma vez que uma máquina esteja disponível a partir desse tempo, a tarefa é alocada àquela que permite o menor tempo de término possível. O cronograma é construído progressivamente, e ao concluir cada tarefa, o decodificador atualiza o grau de entrada de suas sucessoras, adicionando à fila de prontidão aquelas que se tornam aptas para execução.
Caso alguma tarefa não possa ser alocada dentro do intervalo de tempo pré-definido, a solução é considerada inviável e penalizada com um valor de *fitness* elevado.

Função de Aptidão (*Fitness*) Após todas as tarefas serem alocadas, o cronograma resultante é avaliado. O critério de avaliação é o ***makespan*** (C_{max}), definido como o tempo de término da última tarefa no sistema. Esse valor é utilizado diretamente como a função de aptidão:

$$Fitness = C_{max} \tag{1}$$

Soluções com menor *makespan* são preferidas pelo algoritmo.

Figura 7: Cronograma gerado pelo decodificador para o exemplo da Figura 1.



Fonte: O autor (2025).

4.3.4 Geração da População e Operadores Evolutivos

Com o mecanismo de codificação e avaliação definido, o BRKGA aplica seus operadores padrão para evoluir a população:

- **População Inicial:** Uma população de p indivíduos é gerada com chaves aleatórias.
- **Seleção e Cruzamento:** A cada geração, um pai do conjunto de elite e um pai do conjunto não elite são selecionados. Um novo descendente é gerado através do cruzamento enviesado, onde cada gene tem uma probabilidade ρ_e de ser herdado do pai de elite.
- **Mutantes:** Uma porção da nova população é composta por indivíduos mutantes, indivíduos inteiramente novos, e indivíduos gerados aleatoriamente, para garantir a diversidade e evitar a estagnação do algoritmo.

O ciclo se repete por um número definido de gerações, com a expectativa de que o *fitness* da melhor solução melhore progressivamente.

5 Resultados

Este capítulo apresenta os resultados dos experimentos computacionais conduzidos para avaliar o desempenho do algoritmo *Biased Random-Key Genetic Algorithm* (BRKGA) no problema de sequenciamento de tarefas em máquinas paralelas com restrições de precedência. Os experimentos foram divididos em duas etapas principais:

1. **Análise de Sensibilidade de Parâmetros**: calibração dos principais parâmetros do BRKGA utilizando uma instância de tamanho médio.

2. **Análise de Estabilidade e Escalabilidade**: aplicação da melhor configuração de parâmetros em instâncias de diferentes tamanhos, com foco na robustez e custo computacional.

5.1 Configuração Experimental

Os experimentos foram realizados em um computador com um processador Intel I5-11300H e 8GB de memória RAM. O algoritmo foi implementado em C++ com paralelismo via OpenMP.

5.1.1 Instância para Calibração

Utilizou-se uma instância com 100 tarefas e 5 máquinas, com duração total de 1500 unidades de tempo. Esta instância foi baseada na estrutura de instâncias *benchmark* proposta por Taillard(15), que tipicamente contêm informações sobre o número de tarefas, máquinas e tempos de processamento. Para o presente trabalho, a estrutura original foi usada como ponto de partida, mas adaptada para o contexto de máquinas paralelas com precedência. Especificamente, para criar as restrições de precedência, a sequência de operações de um *job* do problema *flowshop* foi utilizada como um molde: cada operação de Taillard(15) foi convertida em uma tarefa independente, e a ordem sequencial original deu origem a uma cadeia de precedência entre essas novas tarefas.

5.1.2 Parâmetros Testados

A Tabela 3 apresenta os valores considerados para cada parâmetro do BRKGA. Cada combinação foi avaliada em 10 execuções independentes. Além dos parâmetros listados na Tabela 3, a implementação do BRKGA utilizada também faz uso dos parâmetros adicionais apresentados na Tabela 4. A escolha dos valores para X_INTVL, X_NUMBER e MAX_GENS foi baseada em configurações e exemplos de uso presentes na documentação oficial da BRKGAAPI, desenvolvida por Resende e Toso(21). Para os parâmetros K (número de populações independentes) e $MAXT$ (número máximo de *threads* para decodificação paralela), os valores foram ajustados com base em testes empíricos preliminares realizados no ambiente computacional deste estudo, buscando otimizar a exploração e o paralelismo, respectivamente.

Tabela 3: Parâmetros e seus respectivos valores considerados para a análise de sensibilidade.

Parâmetro	**Valores Testados**
Tamanho da População (p)	{200; 500; 1000}
Porcentagem da Elite (p_e)	{0,10; 0,15; 0,20}
Porcentagem de Mutantes (p_m)	{0,10; 0,15; 0,20}
Probabilidade de Herança da Elite (ρ_e)	{0,75; 0,85; 0,90}

Fonte: O autor (2025).

Tabela 4: Parâmetros adicionais de implementação do BRKGA e seus valores utilizados.

Parâmetro	**Descrição**	**Valor Utilizado**
K	Número de populações independentes	4
$MAXT$	Número máximo de *threads* para decodificação paralela	8
X_INTVL	Intervalo de gerações para troca de indivíduos elite	100
X_NUMBER	Quantidade de indivíduos elite a serem trocados	2
MAX_GENS	Número máximo de gerações	1000

Fonte: O autor (2025).

5.2 Otimização de Parâmetros

O total de experimentos realizados na otimização dos hiperparâmetros foram 10 execuções independentes para cada combinação de parâmetros apresentada na Tabela 3, resultando em $10 \times 3 \times 3 \times 3 \times 3 = 810$ experimentos computacionais. Os resultados foram compilados e analisados para identificar a configuração mais eficaz. O desempenho de cada configuração foi avaliado observando o valor médio do *makespan*, tempo médio de execução e no *makespan* mínimo (respectivamente).

5.2.1 Desempenho das Melhores Configurações

Devido a quantidade de experimentos executados, a Tabela 5 apresenta as 10 configurações de parâmetros que obtiveram os melhores resultados, os quais estão ordenados pelo valor médio do *makespan* (C_{max}). As primeiras colunas desta tabela apresentam os valores de cada parâmetro, e para cada configuração, são exibidos o valor médio, mínimo, e desvio padrão do *makespan*, além do tempo médio de execução do algoritmos, em milissegundos.

Tabela 5: As 10 melhores configurações de parâmetros encontradas.

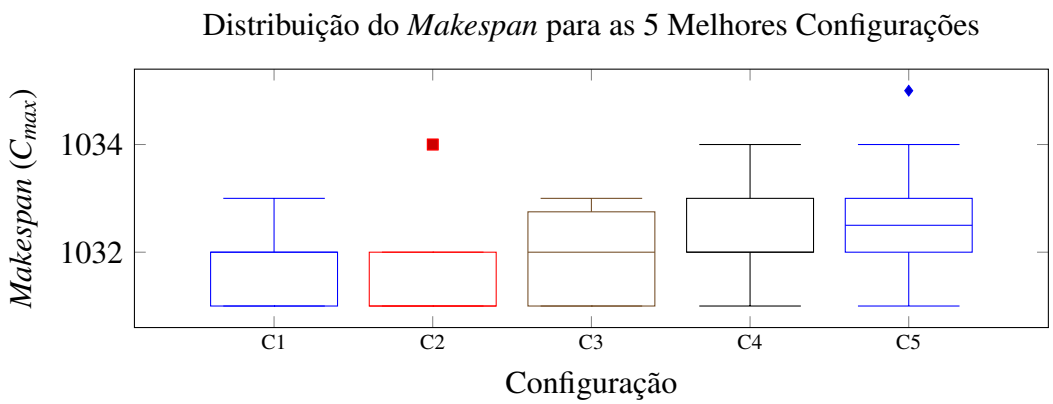
Configuração	**População** (p)	**% Elite** % (p_e)	**% Mutantes** (p_m)	**Prob. Herança** (ρ_e)	**Média de** C_{max}	**Mínimo de** C_{max}	**Desvio Padrão de** C_{max}	**Tempo médio (ms)**
C1	1000	0,20	0,20	0,75	1031,7	1031	0,64	13773,7
C2	1000	0,20	0,15	0,75	1031,8	1031	1,17	13669,5
C3	1000	0,20	0,10	0,75	1031,9	1031	0,83	13885,8
C4	1000	0,15	0,15	0,75	1032,5	1031	0,92	14449,5
C5	1000	0,15	0,10	0,75	1032,7	1031	1,10	14625,8
C6	1000	0,15	0,20	0,75	1033,0	1031	1,67	14467,5
C7	500	0,20	0,20	0,75	1035,5	1032	3,01	6773,38
C8	1000	0,10	0,15	0,75	1035,6	1032	2,58	15186,9
C9	1000	0,10	0,20	0,75	1035,8	1032	3,54	15295,8
C10	1000	0,10	0,10	0,75	1036,3	1032	2,10	15121,8

Fonte: O autor (2025).

A partir da Tabela 5, observa-se que as configurações com melhor desempenho consistentemente utilizam uma população maior ($p = 1000$) e uma probabilidade de herança elite de 75%. É notável que os melhores resultados tenham convergido para o limite superior dos valores testados para o tamanho da população, o que sugere que valores ainda maiores poderiam, potencialmente, levar a soluções de melhor qualidade. A combinação de parâmetros $p = 1000$, $p_e = 20\%$, $p_m = 20\%$ e $\rho_e = 75\%$ apresentou a menor média de *makespan*, com um valor de $1031,7$. Portanto, a combinação $C1$ foi escolhida para as etapas seguintes de análise. A seleção se justifica pelo fato de esta configuração ter alcançado não apenas a menor média de *makespan*, mas também um baixo desvio padrão, o que aponta para a consistência e estabilidade da solução encontrada.

Para uma análise mais detalhada da distribuição dos resultados, a Figura 8 apresenta um *boxplot* comparativo para as 5 melhores configurações identificadas na Tabela 5, rotuladas de $C1$ a $C5$, onde $C1$ corresponde à primeira configuração da tabela, $C2$ à segunda, e assim por diante. Este gráfico permite visualizar a mediana, quartis e possíveis *outliers* (valores discrepantes) do *makespan* para cada uma dessas configurações, complementando a análise de média e desvio padrão.

Figura 8: *Boxplot* comparativo do *makespan* para as 5 melhores configurações de parâmetros.



Fonte: O autor (2025).

5.2.2 Análise da Estabilidade dos Resultados

Com a configuração de parâmetros otimizada definida na seção anterior ($p = 1000$, $p_e = 0,20$, $p_m = 0,20$ e $\rho_e = 0,75$), o algoritmo BRKGA foi avaliado em instâncias de diferentes tamanhos para verificar sua estabilidade e escalabilidade. Dado o comportamento aleatório/probabilístico do BRKGA, foram realizadas 10 execuções independentes do algoritmo, utilizando diferentes sementes para garantir a robustez dos resultados.
As instâncias utilizadas para esta análise, abrangendo diferentes portes, foram geradas seguindo a mesma metodologia descrita na Seção 5.1.1, ou seja, adaptando a estrutura dos *benchmarks* de Taillard(15) para criar um problema de máquinas paralelas com restrições de precedência. As características de cada instância são as seguintes:

- **Instância Pequena**: 3 Máquinas, 1000 unidades de tempo (horizonte), 20 Tarefas.
- **Instância Média**: 5 Máquinas, 1500 unidades de tempo (horizonte), 100 Tarefas.
- **Instância Grande**: 10 Máquinas, 1500 unidades de tempo (horizonte), 250 Tarefas.

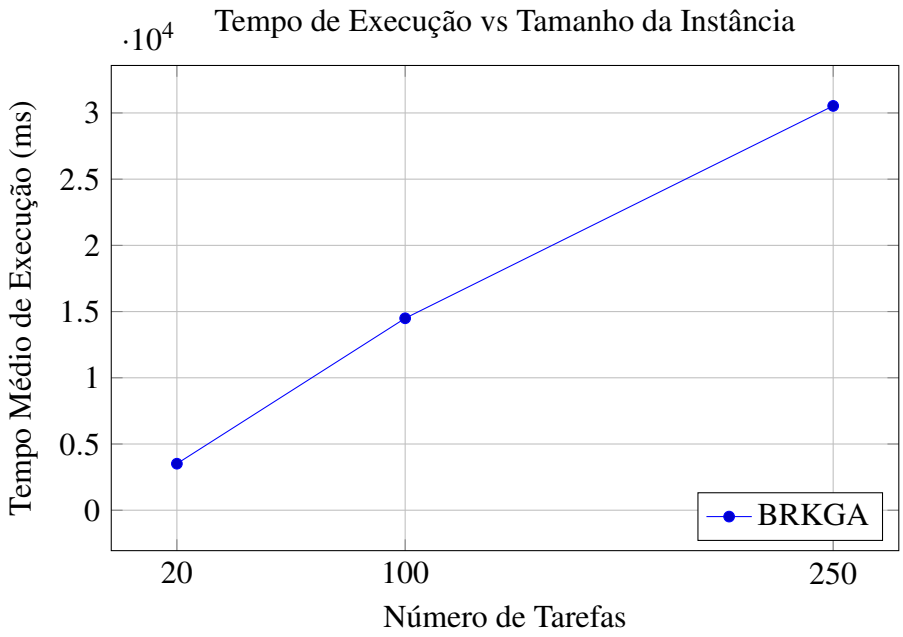
O horizonte de tempo (1000 ou 1500 unidades) para essas instâncias tem pouca relevância no desempenho do algoritmo, os tempos de processamento das tarefas são geralmente muito menores que o horizonte total disponível.
O desempenho do algoritmo foi avaliado com base na média e no desvio padrão do *makespan* (C_{max}), além do tempo médio de execução para cada tipo de instância.

Tabela 6: Resultados da análise de estabilidade e escalabilidade para as diferentes instâncias.

Instância	**Média C_{max}**	**Mínimo C_{max}**	**Desvio Padrão C_{max}**	**Tempo médio (ms)**
Pequena (20 tarefas)	343	343	0	3512,1
Média (100 tarefas)	1031,7	1031	0,64	14493,6
Grande (250 tarefas)	10039,4	10038	0,66	30531,7

O autor (2025).

Figura 9: Tempo médio de execução do BRKGA conforme o tamanho da instância.



Fonte: O autor (2025).

A Tabela 6 apresenta os resultados obtidos para as diferentes instâncias, demonstrando o comportamento do BRKGA frente ao aumento da complexidade do problema. Para a Instância Pequena (20 tarefas), o algoritmo exibe uma ótima estabilidade, com desvio padrão do C_{max} igual a 0, indicando que a solução ótima ou muito próxima da ótima é encontrada consistentemente em todas as 10 execuções. O tempo médio de execução para esta instância é de 3512,1 ms, o que é bem rápido para a dimensão do problema.

À medida que o tamanho das instâncias aumenta para um tamanho médio com 100 tarefas, e grande com 250 tarefas, observa-se que o algoritmo consegue manter um excelente nível de estabilidade. Os desvios padrão alcançados do C_{max} são de 0,64 e 0,66; respectivamente. Tais valores são baixos, sugerindo que o BRKGA converge para soluções de qualidade similar em diferentes execuções, mesmo em problemas com maior número de tarefas e máquinas.

No entanto, como esperado, o custo computacional aumenta significativamente com a complexidade da instância. O tempo médio de execução passa para 14493,6 ms (um aumento de aproximadamente 312,7% em relação à instância pequena) considerando a instância de tamanho médio, e atinge 30531,7 ms (um aumento de aproximadamente 769,3% em relação à instância pequena) na instância grande. Esse comportamento é natural para algoritmos de otimização que exploram um espaço de busca combinatório que cresce consideravelmente com o tamanho do problema. Apesar do aumento do tempo gasto pelo BRKGA para resolver o problema, considerando as diferentes instâncias, os tempos de execução ainda são considerados aceitáveis, sendo que a execução independente com maior duração para a instância grande foi de 34386,6 ms. Além disso, pode-se perceber a estabilidade e eficiência do

BRKGA em solucionar instâncias de diferentes tamanho, mesmo para cenários mais desafiadores, isso reforça sua robustez para resolver o problema de sequenciamento de tarefas em máquinas paralelas com restrições de precedência.

Resumindo, os experimentos detalhados neste capítulo validaram a eficácia do algoritmo para o problema de sequenciamento de tarefas em máquinas paralelas idênticas com restrições de precedência. A análise de sensibilidade permitiu identificar a melhor configuração de parâmetros, que, quando aplicada às instâncias de diferentes tamanhos, demonstrou a capacidade do algoritmo em encontrar soluções de alta qualidade com notável estabilidade, mesmo diante do aumento da complexidade e do consequente incremento no custo computacional.

6 Considerações Finais

Este trabalho teve como objetivo principal desenvolver e avaliar uma solução baseada no algoritmo *Biased Random-Key Genetic Algorithm* (BRKGA) para o problema de sequenciamento de tarefas em máquinas paralelas idênticas com restrições de precedência ($P|prec|C_{max}$). A motivação para essa pesquisa reside na crescente complexidade dos ambientes industriais e na necessidade de otimização dos processos produtivos para minimizar o *makespan* e maximizar a eficiência global.

A metodologia adotada nos experimentos computacionais foi dividida em duas etapas principais, conforme detalhado no Capítulo 5: *(i)* uma análise de sensibilidade de parâmetros; e *(ii)* uma análise de estabilidade e escalabilidade. A primeira etapa permitiu otimizar a escolha dos parâmetros do BRKGA utilizando uma instância de tamanho médio, identificando a melhor combinação de valores para p, p_e, p_m e ρ_e. Esta etapa foi fundamental para garantir que o algoritmo fosse executado com valores de parâmetros adequados em cenários posteriores com instâncias distintas.

Os resultados da análise de estabilidade e escalabilidade, apresentados na Tabela 6, demonstram a robustez e a eficácia do algoritmo. Para a instância pequena, o algoritmo alcançou alta estabilidade, com desvio padrão nulo referente ao valor de C_{max} (*makespan*), indicando que a solução ideal foi consistentemente encontrada em todas as execuções independentes. Para as instâncias média e grande, apesar do aumento esperado no tempo computacional, os resultados obtidos mantiveram um excelente nível de estabilidade, com baixos valores de desvios padrão do C_{max} (0,64 e 0,66, respectivamente). Este comportamento reforça a capacidade do algoritmo em convergir para soluções de alta qualidade de forma consistente, mesmo em problemas de maior tamanho. A crescente demanda por tempo de execução em instâncias maiores é um comportamento natural para problemas combinatórios, classificados como NP-difíceis, evidenciando que, embora o BRKGA seja uma meta-heurística, sua aplicação é viável e promissora para o contexto industrial.

6.1 Contribuições do Trabalho:

- **Adaptação e Aplicação do BRKGA**: Este trabalho adaptou e aplicou com sucesso o *framework* BRKGA a um problema de sequenciamento de tarefas em máquinas paralelas idênticas com precedência, um tipo de problema de grande relevância prática.

- **Metodologia de Calibração**: Foi realizada uma análise de sensibilidade dos valores dos parâmetros, garantindo uma configuração otimizada para o desempenho do algoritmo.

- **Avaliação de Estabilidade e Escalabilidade**: O estudo forneceu uma análise da estabilidade e escalabilidade em diferentes tamanhos de problema, demonstrando sua robustez e aplicabilidade a cenários industriais realistas.

6.2 Limitações do Trabalho

Apesar dos resultados promissores, é importante reconhecer algumas limitações. O modelo atual foca apenas na minimização do *makespan*. Embora essa seja uma métrica crucial, ambientes industriais reais frequentemente envolvem múltiplos objetivos, como minimização de custos, atrasos de entrega e utilização de recursos. Adicionalmente, uma limitação reside na geração das instâncias de teste. Uma vez que não há um conjunto de *benchmarks* amplamente difundido para o problema específico de máquinas paralelas com o tipo de precedência aqui estudado, optou-se por adaptar a estrutura das conhecidas instâncias de Taillard(15). Embora essa abordagem garanta um ponto de partida estruturado e desafiador, a conversão da lógica sequencial do *flowshop* em restrições de precedência resulta em grafos com uma topologia específica (cadeias de dependência), que podem não representar toda a variedade de cenários práticos com estruturas de precedência mais complexas.

6.3 Perspectivas para Trabalhos Futuros

Com base nas contribuições e limitações identificadas, existem diversas possibilidades de pesquisa que podem ser exploradas futuramente:

- **Aplicação de abordagens Multi-objetivo**: Estender o modelo do BRKGA para otimizar simultaneamente o *makespan* e outras métricas relevantes, como o atraso total das tarefas, ou a utilização de máquinas. Isso pode ser feito através da adaptação de abordagens multi-objetivo como o NSGA-II Zhao e Yuan(1).

- **Adição de Novas Restrições**: Incorporar restrições adicionais mais complexas, como janelas de tempo variáveis por tarefa, tempos de *setup* dependentes da sequência ou da máquina, e disponibilidade dinâmica de recursos.

- **Tipos de Máquinas Heterogêneas**: Explorar o problema em ambientes com máquinas uniformes (diferentes velocidades) ou não-relacionadas (diferentes capacidades), tornando a aplicação mais generalista.

- **Instâncias Reais ou Mais Complexas**: Testar o algoritmo com conjuntos de dados provenientes de cenários industriais reais ou com instâncias mais diversas em termos de topologia do grafo de precedência e distribuição dos tempos de processamento.

- **Análise de Parâmetros Ampliada:** Investigar o impacto de parâmetros fora do intervalo originalmente testado, com especial atenção ao tamanho da população. Uma vez que os melhores resultados da análise de sensibilidade foram consistentemente obtidos com o maior valor avaliado ($p = 1000$), uma análise com populações ainda maiores poderia revelar novos ganhos de desempenho, explorando o compromisso entre a qualidade da solução e o aumento do custo computacional.

Este trabalho representa um passo significativo no uso de meta-heurísticas para a otimização de problemas de escalonamento de tarefas, abrindo caminho para futuras pesquisas que busquem soluções ainda mais sofisticadas e aderentes às demandas da Indústria 4.0.

REFERÊNCIAS

1 ZHAO, Z.; YUAN, Q. Integrated multi-objective optimization of predictive maintenance and production scheduling: Perspective from lead time constraints. **Journal of Intelligent Management Decision**, p. 67–77, 2022.

2 MZILI, T. et al. Optimizing production scheduling with the spotted hyena algorithm: A novel approach to the flow shop problem. **Reports in Mechanical Engineering**, p. 90–103, 2023.

3 TANAKA, S.; ARAKI, M. A branch-and-bound algorithm with lagrangian relaxation to minimize total tardiness on identical parallel machines. **International Journal of Production Economics**, p. 446–458, 2008.

4 RESENDE, M. G. C.; GONçALVES, J. F. Biased random-key genetic algorithms for combinatorial optimization. **Journal of Heuristics**, p. 1–22, 2011.

5 AGáRDI, A.; NEHéZ, K. The unrelated parallel machines scheduling problem with machine and job dependent setup times, availability constraints, time windows and maintenance times. **Management and Production Engineering Review**, p. 15—-24, 2023.

6 GHALAMI, L.; GROSU, D. Scheduling parallel identical machines to minimize makespan:a parallel approximation algorithm. **Journal of Parallel and Distributed Computing**, p. 221–231, 2019.

7 FIDELIS, M. B.; ARROYO, J. E. C. Meta-heurísticas para o sequenciamento de famílias de tarefas em máquinas paralelas idênticas de processamento em lote. **Brazilian Journal of Development**, p. 62006—-62027, 2022.

8 LAWLER, E. L. et al. Chapter 9 sequencing and scheduling: Algorithms and complexity. In: **Logistics of Production and Inventory**. Elsevier, 1993, (Handbooks in Operations Research and Management Science, v. 4). p. 445–522. Disponível em: https://www.sciencedirect.com/science/article/pii/S0927050705801896.

9 SCHALLER, J. E. Minimizing total tardiness for scheduling identical parallel machines with family setups. **Computers Industrial Engineering**, v. 72, p. 274–281, 2014. ISSN 0360-8352. Disponível em: https://www.sciencedirect.com/science/article/pii/S0360835214001120.

10 AZEVEDO, A.; LANDGRAF, G.; SCHROEDER, M. Uma heurística para o problema sequenciamento de tarefas com restrições adicionais em um centro de distribuição. p. 1–6, 02 2018.

11 LAHA, D.; BEHERA, D. K. A comprehensive review and evaluation of lpt, multifit, combine and listfit for scheduling identical parallel machines. **International Journal of Information and Communication Technology**, p. 1–15, 2015.

12 RESENDE, M. G. C. et al. Biased random-key genetic algorithms: A review. **European Journal of Operational Research**, p. 1–22, 2024.

13 RUBéN; STüTZLE, T. A simple and effective iterated greedy algorithm for the permutation flowshop scheduling problem. **European Journal of Operational Research**, p. 2033–2049, 2007.

14 NAWAZ, M.; ENSCORE, E. E.; HAM, I. A heuristic algorithm for the m-machine, n-job flow-shop sequencing problem. **Omega**, v. 11, n. 1, p. 91–95, 1983. ISSN 0305-0483. Disponível em: https://www.sciencedirect.com/science/article/pii/0305048383900889.

15 TAILLARD Éric D. Benchmarks for basic scheduling problems. **European Journal of Operational Research**, p. 278–285, 1993.

16 AZIZOğLU, M.; KIRCA Ömer. Tardiness minimization on parallel machines. **International Journal of Production Economics**, p. 163–168, 1998.

17 YALAOUI, F.; CHU, C. Parallel machine scheduling to minimize total tardiness. **International Journal of Production Economics**, p. 265–279, 2002.

18 SHIM, S.; KIM, Y.-D. Scheduling on parallel identical machines to minimize total tardiness. **European Journal of Operational Research**, p. 135—-146, 2007.

19 XAVIER, A. da S.; LINS, S. L. S.; SILVA, A. A. Sequenciamento de tarefas em máquinas paralelas uniformes com técnicas de fluxo em rede: uma aplicação prática. **Brazilian Journal of Development**, p. 26354–26370, 2020.

20 EIBEN, A. E.; SMITH, J. E. **Introduction to Evolutionary Computing**. 2nd. ed. Berlin, Germany: Springer, 2015. ISBN 978-3-662-46601-0.

21 RESENDE, M. G. C.; TOSO, R. F. A c++ application programming interface for biased random-key genetic algorithms. **Optimization Methods Software**, p. 1–16, 2015.